LAPRAK IOT

Digital Output, Digital Input

Modifikasi Sistem Sakelar

Oleh:

NAMA : M. Kurnia Akbar

NPM : 25782101

KELAS : TRI – 2D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

PENDAHULUAN

Latar Belakang

Mikrokontroler ESP32 merupakan sistem dalam kemasan (*System on a Chip*) berbasis prosesor 32-bit yang menawarkan kecepatan pemrosesan tinggi, efisiensi daya, serta integrasi Wi-Fi dan Bluetooth. Keunggulan ini menjadikan ESP32 sebagai standar populer dalam pengembangan proyek *embedded system* dan *Internet of Things* (IoT).

Tujuan Praktikum

- Menerapkan logika *state tracking* untuk menyimpan kondisi output (*ON/OFF*) pada ESP32.
- Membedakan perilaku sakelar *momentary* dan *latching*.
- Menerapkan teknik *software debouncing* berbasis delay untuk mencegah *flickering* akibat pantulan tombo

Alat dan bahan

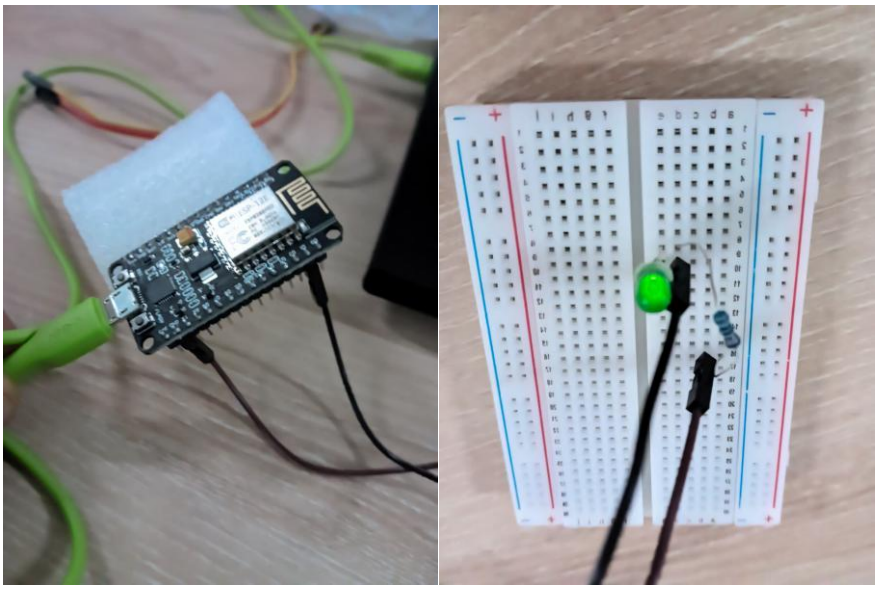
- 1x ES8266 DevKit v4
- 1x Push Button
- 1x LED Merah
- 1x Resistor 10 kΩ (*Pull-down*)
- 1x Resistor 200 Ω (*Pembatas arus LED*)
- Kabel jumper secukupnya

PRAKTIKUM 1
DIGITAL OUTPUT (MENYALAKAN LED)

Tujuan

Praktikum ini bertujuan memprogram NodeMCU agar mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal.

Rangkaian



Langkah Kerja

1. Pasang NodeMCU pada *breadboard*.
2. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
    - Kaki yang lebih panjang adalah Anoda (Positif).
    - Kaki yang lebih pendek (dan terdapat sisi datar di tepi plastik kepala LED) adalah Katoda (Negatif).
3. Hubungkan pin D1 (GPIO 5) dari NodeMCU ke kaki Anoda LED menggunakan kabel *jumper*.
4. Hubungkan kaki Katoda LED ke salah satu ujung Resistor 220 Ohm.
5. Hubungkan ujung Resistor lainnya ke jalur GND pada NodeMCU. *(Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar)*

Kode Program

```
const int ledPin = 5;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  Serial.println("Praktikum 1 - Digital Output Dimulai!");
}

void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("LED Menyala");
  delay(1000);

  digitalWrite(ledPin, LOW);
  Serial.println("LED Mati");
  delay(1000);
}
```

Cara Kerja Program

Program ini bekerja dengan mendaftarkan pin D1 (GPIO 5) pada NodeMCU sebagai jalur OUTPUT listrik dan mengaktifkan komunikasi serial berkecepatan 115200 baud pada fungsi setup() yang hanya berjalan sekali saat perangkat menyala. Selanjutnya, fungsi loop() menjalankan siklus pengulangan tanpa henti yang diawali dengan memberi tegangan 3.3V (HIGH) ke pin D1 sehingga arus listrik mengalir melewati LED dan resistor ke jalur GND untuk menyalakan LED serta mengirim teks "LED Menyala" ke Serial Monitor, lalu menahannya selama 1000 milidetik (1 detik) via fungsi delay(1000). Setelah itu, program memutus tegangan (LOW) menjadi 0V untuk memadamkan LED dan mencetak teks "LED Mati" dengan jeda waktu yang sama, sebelum akhirnya mengulang seluruh proses tersebut dari awal secara terus-menerus.

Hasil

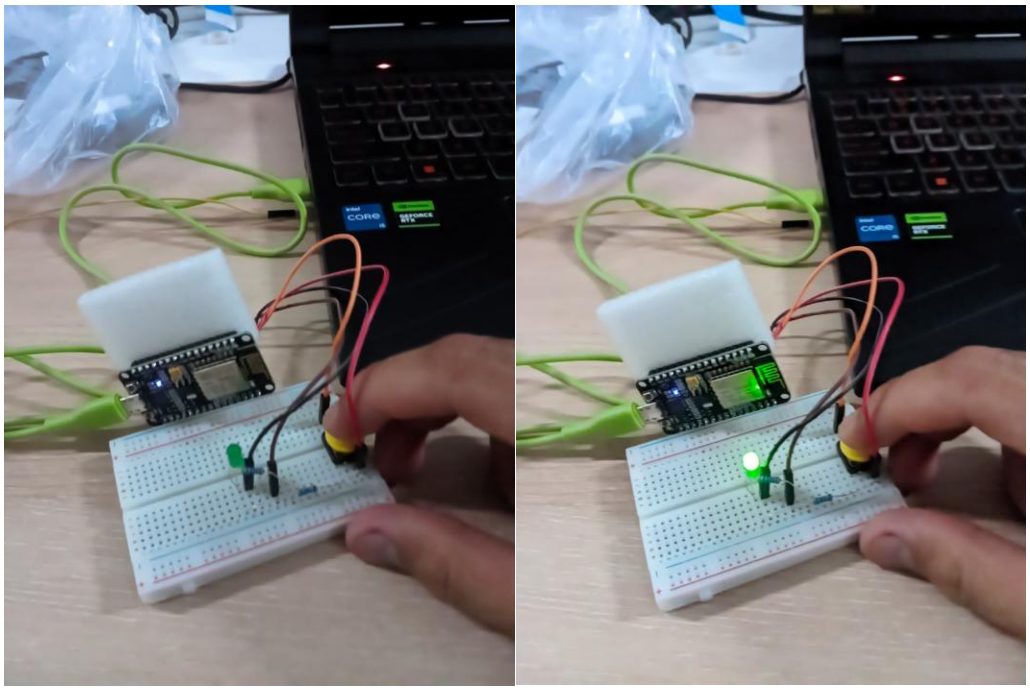
Hasil pengujian menunjukkan bahwa program berhasil berjalan secara sinkron antara komponen fisik dan perangkat lunak, di mana LED pada breadboard berkedip terang selama 1 detik dan padam selama 1 detik secara terus-menerus. Di saat yang sama, Serial Monitor pada Arduino IDE (dengan kecepatan 115200 baud) mencetak pesan pembuka "Praktikum 1 - Digital Output Dimulai!" satu kali di awal, yang kemudian diikuti dengan cetakan teks status "LED Menyala" dan "LED Mati" secara bergantian tepat sesuai dengan perubahan kondisi menyala dan padamnya LED fisik secara *real-time*.

PRAKTIKUM 2 –
DIGITAL INPUT (MEMBACA PUSH BUTTON)

Tujuan

Praktikum ini bertujuan mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor *pull-down* eksternal.

Rangkaian



Langkah Kerja

1. Biarkan rangkaian LED pada pin D1 dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan kaki pertama *push button* ke pin 3V3 (3.3V) pada NodeMCU.
4. Hubungkan kaki kedua *push button* ke pin D2 (GPIO 4) NodeMCU.
5. Pada kaki kedua yang sama, pasang sebuah Resistor 10k Ohm, lalu hubungkan ujung lain dari resistor tersebut ke jalur GND NodeMCU.

Kode Program

```
const int buttonPin = 4;
const int ledPin = 5;

int buttonState = 0;

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  buttonState = digitalRead(buttonPin);

  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Tombol ditekan! -> LED ON");
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

Cara Kerja Program

Program ini bekerja dengan mengonfigurasi pin D2 (GPIO 4) sebagai INPUT digital dan pin D1 (GPIO 5) sebagai OUTPUT pada fungsi setup(), lalu secara berulang membaca status tegangan pada pin D2 menggunakan digitalRead() di dalam loop(). Berkat bantuan resistor *pull-down* 10k Ω, pin D2 berada pada logika LOW (0V) saat tombol tidak ditekan sehingga LED tetap padam, namun ketika tombol ditekan, aliran arus 3.3V mengubah statusnya menjadi HIGH yang memicu fungsi if untuk menyalakan LED dan mencetak pesan "Tombol ditekan! -> LED ON" ke Serial Monitor. Hasil pengujian menunjukkan sistem bekerja secara responsif dan *real-time*, di mana LED fisik menyala terang selaras dengan kemunculan teks status di Serial Monitor (115200 baud) saat tombol ditekan dan ditahan, lalu seketika LED padam serta pengiriman teks terhenti begitu tombol dilepaskan.

Hasil

Hasil pengujian menunjukkan bahwa ketika tombol ditekan dan ditahan, LED fisik seketika menyala terang diiringi kemunculan teks "Tombol ditekan! -> LED ON" secara berulang pada Serial Monitor (115200 baud), dan ketika tombol dilepaskan, LED fisik langsung padam serta pengiriman teks pada Serial Monitor terhenti sepenuhnya.

LATIHAN

Latihan 1 – Pemahaman Logika Pin

Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Jawaban: Jika Anda menyambungkan tombol dengan logika *Active-High* (*pull-down*) ke pin D8 (GPIO 15) dan menekannya saat NodeMCU baru diberi daya, NodeMCU akan mengalami *boot failure* atau *stuck* saat *startup*. Hal ini terjadi karena pin D8 merupakan pin *boot strapping* khusus yang secara mutlak wajib berada dalam kondisi *LOW* (0V) saat proses *power-on* atau *reset* agar mikrokontroler ESP8266 dapat masuk ke mode *booting* dari memori *flash* internal. Ketika tombol ditekan saat daya dinyalakan, arus 3.3V dari rangkaian *pull-down* memaksa sinyal pin D8 menjadi *HIGH*, sehingga melanggar syarat fisik *startup* dan menyebabkan mikrokontroler kebingungan memilih *boot mode* hingga sistem akhirnya *freeze* sebelum program sempat dijalankan.

Latihan 2 (Implementasi Kode)

Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi delay() yang harus Anda ubah.

```C++
digitalWrite(ledPin, HIGH); // LED Menyala
delay(200);                 // Nyala cepat selama 200 ms

digitalWrite(ledPin, LOW);  // LED Mati
delay(800);                 // Mati agak lama selama 800 ms
```

Latihan 3 (Analisis Sirkuit)

Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Jawaban: Jika kaki LED dipasang terbalik (Anoda ke GND dan Katoda ke D1), program tidak akan mengalami error kompilasi karena kesalahan tersebut berada pada level sirkuit fisik (*hardware*), bukan pada sintaks kode C/C++. LED sendiri merupakan dioda yang memiliki polaritas dan hanya mengizinkan arus searah mengalir dari Anoda ke Katoda (*forward bias*). Pemasangan Anoda ke GND (0V) dan Katoda ke D1 (3.3V saat HIGH) menyebabkan LED berada dalam kondisi prasikap mundur (*reverse bias*) yang menyumbat aliran arus listrik, sehingga beda potensial yang dihasilkan tidak mampu mengalirkan arus dan membuat LED tetap mati.

Latihan 4 (Pengembangan Algoritma)

Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

```
C++

if (digitalRead(buttonPin) == HIGH) {
  digitalWrite(ledPin, LOW);   // Saat tombol DITEKAN, LED mati
} else {
  digitalWrite(ledPin, HIGH);  // Saat tombol TIDAK ditekan, LED menyala
}
```

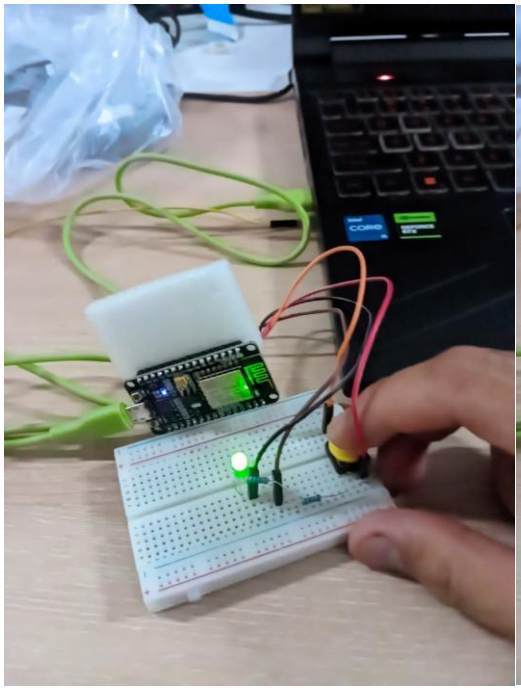
Pada rangkaian *pull-down*, digitalRead(buttonPin) akan bernilai HIGH saat tombol ditekan dan LOW saat tombol dilepas. Dengan membalik perintah digitalWrite(), logika program diubah sehingga ketika tombol ditekan (HIGH), GPIO 5 memutus arus ke LOW untuk mematikan LED. Sebaliknya, saat tombol dilepas (LOW), GPIO 5 secara otomatis mengirimkan sinyal HIGH untuk menyalakan LED

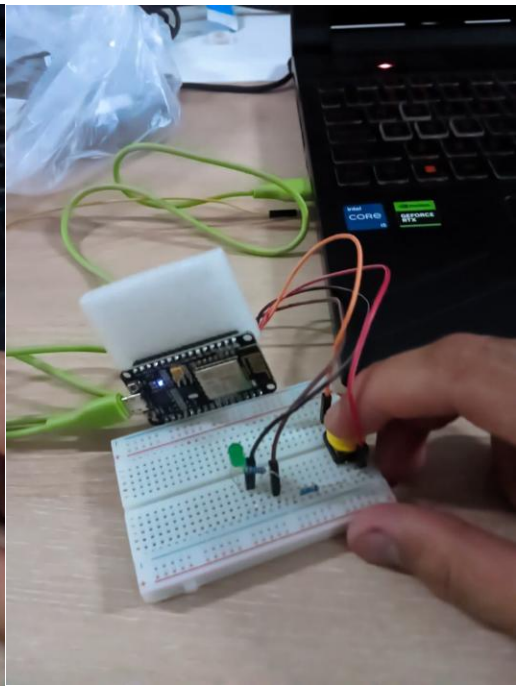
TUGAS
MODIFIKASI SISTEM SAKELAR *TOGGLE* (*LATCHING*)

Tujuan

Tujuan dari program ini adalah mengubah perilaku tombol *momentary* (hanya aktif saat ditahan) menjadi sistem sakelar *latching* (tetap mempertahankan status *ON* atau *OFF* setelah ditekan satu kali). Selain itu, program bertujuan untuk menerapkan teknik *state tracking* dan *software debouncing* agar peralihan status LED berjalan stabil tanpa gangguan pembacaan ganda.

Rangkaian





Kode Program

```
const int buttonPin = 4;
const int ledPin = 5;

int ledState = LOW;
int lastButtonState = LOW;

void setup() {
  Serial.begin(115200);

  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);

  digitalWrite(ledPin, ledState);
}

void loop() {
  int buttonState = digitalRead(buttonPin);

  // Mendeteksi saat tombol baru ditekan
  if (buttonState == HIGH && lastButtonState == LOW) {
    ledState = !ledState;

    digitalWrite(ledPin, ledState);

    if (ledState == HIGH) {
      Serial.println("Tombol ditekan -> LED ON");
    } else {
      Serial.println("Tombol ditekan -> LED OFF");
    }

    delay(50); // debounce sederhana
  }

  lastButtonState = buttonState;
}
```

Penjelasan logika

Pelacakan status (*state tracking*) bekerja dengan menyimpan kondisi LED pada variabel ledState, yang nilainya dibalik menggunakan logika NOT (!ledState) setiap kali deteksi transisi (*edge detection*) mendeteksi tombol baru pertama kali ditekan (currentButtonState == HIGH && lastButtonState == LOW berkat pembanding lastButtonState), sementara instruksi delay(200) berfungsi sebagai *debouncing* untuk memberi jeda agar getaran sinyal palsu dari tombol diabaikan sebelum pembacaan berikutnya.

Hasil Pengujian

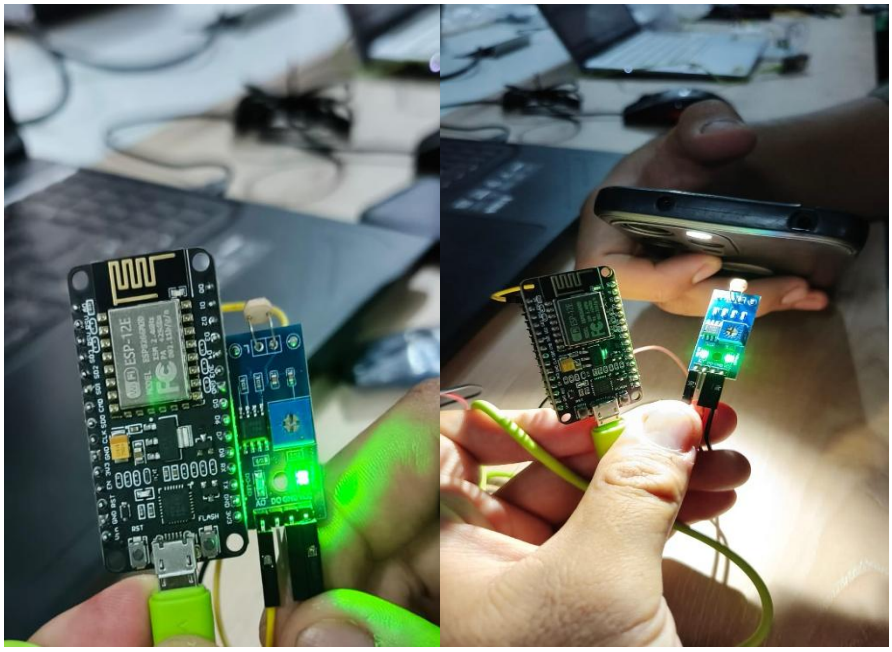
Pengujian menunjukkan bahwa saat mikrokontroler pertama kali diberi daya (*power on*), LED berada dalam kondisi awal mati (LOW), kemudian ketika tombol ditekan satu kali secara singkat lalu dilepas, sinyal mengalami transisi dari LOW ke HIGH yang memicu sistem membalik status ledState menjadi HIGH sehingga LED menyala dan tetap menyala secara stabil, selanjutnya pada penekanan kedua sistem mendeteksi transisi baru dan membalik kembali status ledState menjadi LOW sehingga LED padam dan tetap padam, serta seluruh proses eksekusi berjalan tanpa fenomena *flickering* atau respons acak berkat efektivitas implementasi *delay debounce* selama 200 milidetik.

Praktikum 1:
Pembacaan Intensitas Cahaya (LDR) via ADC

Tujuan

Sensor LDR akan menurunkan hambatannya (resistansi) saat terpapar cahaya terang, dan meningkatkan resistansinya saat gelap. Untuk menerjemahkan perubahan resistansi ini menjadi variasi tegangan yang bisa diukur oleh pin A0, kita menggunakan sirkuit pembagi tegangan (*Voltage Divider*).

Rangkaian



Langkah Kerja

1. Tancapkan satu kaki LDR ke lubang *breadboard* yang terhubung langsung ke pin 3V3 (3.3 Volt) NodeMCU.
2. Tancapkan kaki LDR yang satunya ke sebuah jalur parit kosong di *breadboard*.
3. Hubungkan parit LDR tersebut dengan pin A0 menggunakan sebuah kabel *jumper*.
4. Pada parit LDR yang sama (sejajar dengan kabel ke A0), pasang salah satu kaki Resistor 10k Ohm.
5. Hubungkan ujung hulu Resistor 10k Ohm tersebut ke rel GND.

Kode Program

```
const byte ldrPin = A0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int ldrValue = analogRead(ldrPin);

  Serial.print("Intensitas Cahaya (ADC): ");
  Serial.println(ldrValue);

  delay(1000);
}
```

Penjelasan Logika

Program bekerja dengan memanfaatkan fungsi pembacaan analog pada mikrokontroler NodeMCU untuk memantau perubahan tegangan yang dihasilkan oleh sirkuit pembagi tegangan (*voltage divider*) antara LDR dan resistor 10k Ohm. Melalui instruksi analogRead(ldrPin), sinyal analog murni berupa tegangan konstan antara 0V hingga 3.3V yang diterima pada pin A0 dikonversi oleh *Analog-to-Digital Converter* (ADC) internal 10-bit menjadi nilai digital berskala 0 hingga 1023. Pembacaan ini diperbarui setiap 1000 milidetik (delay(1000)) dan dikirimkan ke komputer via komunikasi Serial dengan kecepatan *baud rate* 115200 untuk ditampilkan pada Serial Monitor.

Hasil Pengujian

Saat disorot cahaya terang menggunakan flash ponsel, resistansi LDR menurun drastis sehingga arus listrik dari jalur 3.3V mengalir menuju pin A0 dan menghasilkan tegangan mendekati 3.3V dengan pembacaan nilai ADC pada Serial Monitor membesar mendekati nilai maksimum 1023, sedangkan saat kondisi gelap akibat tertutup rapat jari, resistansi LDR meningkat sangat tinggi menahan laju arus sehingga pin A0 tertarik ke jalur GND melalui resistor 10k Ohm yang menurunkan tegangan mendekati 0V dan menghasilkan nilai pembacaan ADC yang menurun drastis mendekati angka 0.

```
Intensitas Cahaya (ADC): 986
Intensitas Cahaya (ADC): 991
Intensitas Cahaya (ADC): 999
Intensitas Cahaya (ADC): 997
Intensitas Cahaya (ADC): 997
Intensitas Cahaya (ADC): 950
Intensitas Cahaya (ADC): 941
Intensitas Cahaya (ADC): 953
Intensitas Cahaya (ADC): 926
Intensitas Cahaya (ADC): 943
Intensitas Cahaya (ADC): 969
Intensitas Cahaya (ADC): 1013
Intensitas Cahaya (ADC): 969
Intensitas Cahaya (ADC): 1010
Intensitas Cahaya (ADC): 1013
Intensitas Cahaya (ADC): 1016
```

```
Intensitas Cahaya (ADC): 61
Intensitas Cahaya (ADC): 75
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 67
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 66
Intensitas Cahaya (ADC): 70
Intensitas Cahaya (ADC): 68
```

Praktikum 2
Membaca Data Sensor Suhu & Kelembapan (DHT)

Tujuan

Sensor DHT menggunakan protokol komunikasi digital *single-wire*. Artinya, seluruh data (suhu dan kelembapan) ditransmisikan melalui satu kabel data saja. Protokol ini sangat bergantung pada waktu (*timing*). Mikrokontroler harus mengirimkan perintah "minta data" dengan durasi *HIGH* dan *LOW* yang sangat presisi. Setelah itu, sensor akan mengirimkan kembali serangkaian pulsa digital yang panjangnya spesifik menunjukkan nilai suhu dan kelembapan.

Rangkaian



Kode Program

```
#include <DHT.h>

const byte dhtPin = 13;
#define DHTTYPE DHT11

DHT dht(dhtPin, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println("Memulai Sensor Lingkungan...");
}

void loop() {
  delay(2500);

  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  if (isnan(temp) || isnan(hum)) {
    Serial.println("Gagal membaca data dari sensor DHT!");
    return;
  }

  Serial.print("Suhu: ");
  Serial.print(temp);
  Serial.print(" Celcius | Kelembapan: ");
  Serial.print(hum);
  Serial.println(" %");
}
```

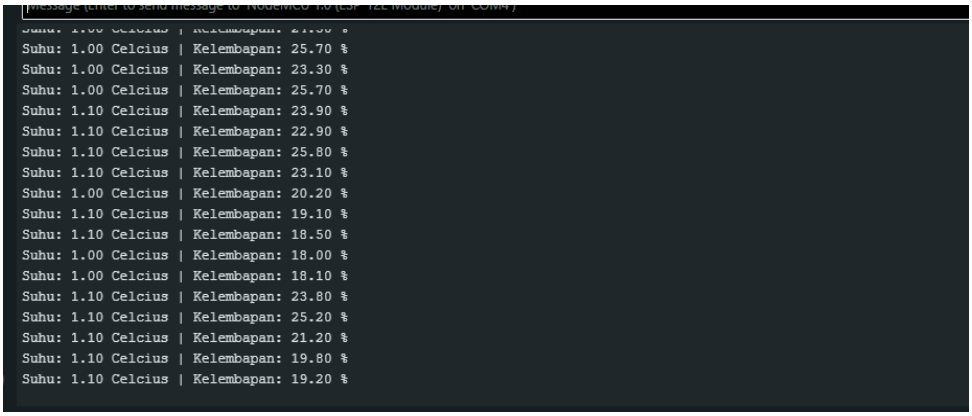
Penjelasan logika

Program memanfaatkan *library* DHT.h untuk mengelola protokol komunikasi digital *single-wire* yang membutuhkan pewaktuan (*timing*) presisi tinggi pada pin GPIO 13 (D7). Pada tahap inisialisasi, objek dht dibentuk dan diaktifkan melalui dht.begin(), serta komunikasi Serial disiapkan pada *baud rate* 115200. Setiap siklus 2,5 detik (delay(2500)), program memanggil fungsi dht.readTemperature() dan dht.readHumidity() untuk mengondisikan mikrokontroler mengirimkan pulsa pemicu dan menerima paket data digital serial yang dikonversi menjadi tipe data desimal (float). Sebelum menampilkan data, fungsi penanganan kesalahan isnan() mengecek validitas nilai yang terbaca; jika terjadi gangguan transmisi atau kabel terlepas, sistem secara otomatis mencetak pesan error dan menghentikan eksekusi loop sementara via perintah return;, sedangkan jika data valid, nilai suhu (°C) dan kelembapan (%) akan dicetak secara terstruktur ke Serial Monitor.

Hasil Pengujian

Saat sensor terhubung dengan benar pada lingkungan normal, Serial Monitor menampilkan pembacaan suhu dan kelembapan udara secara stabil setiap 2,5 detik, yang nilainya akan bertambah secara bertahap ketika sensor diberi hembusan napas

hangat atau didekatkan ke sumber panas dan lembab, serta saat kabel data pada pin D7 dilepas untuk mensimulasikan kegagalan koneksi, pembacaan variabel menghasilkan nilai *Not-A-Number* sehingga sistem langsung memicu blok if (isnan(...)) dan mencetak pesan "Gagal membaca data dari sensor DHT!" tanpa menghentikan program secara keseluruhan.



Latihan 1: Pemahaman Sistem ADC
Batas maksimal nilai yang terbaca adalah **1023** karena *Analog-to-Digital Converter* (ADC) pada pin A0 NodeMCU memiliki **resolusi 10-bit**.

Dalam sistem biner (sistem basis 2 komputer), kapasitas 10-bit dapat membagi rentang tegangan masukan analog (0V hingga 3.3V) menjadi 2^{10} atau **1024 tingkat nilai yang berbeda**. Karena perhitungan data digital selalu dimulai dari angka 0, maka rentang nilai pembacaan yang dihasilkan bergerak dari angka terkecil **0** (mewakili 0 Volt / gelap total) hingga angka terbesar **1023** (mewakili 3.3 Volt / terang maksimal).

Latihan 2: Implementasi Algoritma
Rumus Matematika / Perhitungan Persentase:
Untuk mengubah nilai ADC murni (rentang 0–1023) menjadi skala persentase (0%–100%), rumus yang digunakan adalah membagi nilai pembacaan sensor dengan nilai maksimum ADC (1023), lalu mengalikannya dengan 100.

$$\text{Persentase Intensitas Cahaya (\%)} = \left(\frac{\text{Nilai ADC LDR}}{1023}\right) \times 100$$

Langkah Penjelasan Algoritma:

1. Mikrokontroler membaca nilai mentah analog dari sensor (misal: terbaca 512).
2. Nilai tersebut dibagi dengan nilai batas maksimal ADC, yaitu 1023 ($512/1023 \approx 0{,}5$).
3. Hasil pembagian desimal dikalikan dengan 100 untuk mendapatkan persentase ($0{,}5 \times 100 = 50\%$).

Latihan 3: Analisis Rangkaian

Pada modul Relay bertipe Active-Low, mekanisme sakelar internal relay akan aktif (menghubungkan arus) saat menerima perintah sinyal tegangan rendah (LOW), dan akan mati/nonaktif saat menerima perintah sinyal tegangan tinggi (HIGH).

Jika program mengeksekusi instruksi HIGH pada pin relay:

- Status Relay: Relay akan berada dalam kondisi mati / nonaktif (OFF).
- Status Lampu Ruangan:
    - Jika kabel lampu dipasang pada terminal Normally Open (NO), maka lampu akan PADAM (OFF).
    - Jika kabel lampu dipasang pada terminal Normally Closed (NC), maka lampu akan MENYALA (ON).

Latihan 4: Pengembangan Logika

Alur Kerja Logika Kendali LED Otomatis:

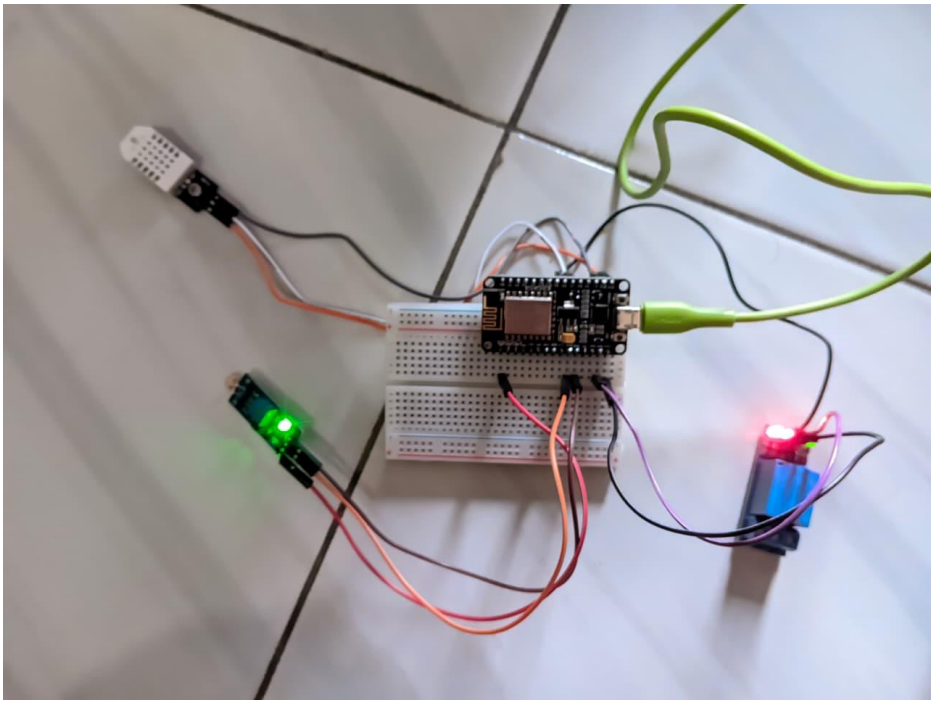
1. Pembacaan Sensor: Mikrokontroler secara terus-menerus membaca data intensitas cahaya dari pin A0 tempat sensor LDR terhubung.
2. Pengecekan Kondisi (If-Else):
    - Kondisi Gelap: Sistem mengecek apakah nilai ADC yang terbaca berada di bawah angka 200. Jika *YA* (berarti kondisi lingkungan sangat gelap), mikrokontroler mengirimkan sinyal HIGH ke pin D1 untuk menyalakan LED.
    - Kondisi Terang: Jika nilai ADC berada sama dengan atau di atas angka 200 (kondisi lingkungan cukup terang), sistem mengeksekusi blok pembalik dan mengirimkan sinyal LOW ke pin D1 untuk mematikan LED.
3. Pewaktuan: Proses pembacaan dan evaluasi kondisi ini diulang secara berkala dengan penundaan waktu singkat (misalnya setiap 1 detik).

Tugas
Sistem Otomasi Kontrol Lokal Terpadu

Tujuan

Tujuan dari program ini adalah mengimplementasikan konsep *Edge Computing* pada sistem *Smart Warehouse* dengan memadukan akuisisi data multi-sensor (suhu digital dari DHT11 dan intensitas cahaya analog dari LDR) untuk mengeksekusi aktuator otomatis (Relay dan LED) secara mandiri berbasis aturan (*rule engine*) tanpa bergantung pada koneksi internet.

Rangkaian



Kode Program

```
// the setup function runs once when you press reset
#include <DHT.h>

#define DHTPIN D7
#define DHTTYPE DHT22

const int ldrPin = A0;
const int relayPin = D6;
const int ledPin = D1;

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);

  dht.begin();

  pinMode(relayPin, OUTPUT);
  pinMode(ledPin, OUTPUT);

  // Kondisi awal: aktuator mati
  digitalWrite(relayPin, HIGH);
  digitalWrite(ledPin, LOW);

  Serial.println("Sistem Monitoring Dimulai");
}

void loop() {
  float suhu = dht.readTemperature();
  int ldrValue = analogRead(ldrPin);

  // Cek apakah pembacaan DHT valid
  if (isnan(suhu)) {
    Serial.println("Gagal membaca DHT22!");
    delay(2000);
    return;
  }

  Serial.print("Suhu: ");
  Serial.print(suhu);
  Serial.print(" C | LDR: ");
  Serial.println(ldrValue);

  // Suhu > 34 ATAU LDR < 300
  if (suhu > 40|| ldrValue < 500) {

    // Untuk relay Active-Low
    digitalWrite(relayPin, LOW);
    digitalWrite(ledPin, HIGH);

    Serial.println("Peringatan: Aktuator Aktif!");

  } else {

    // Relay mati
    digitalWrite(relayPin, HIGH);
    digitalWrite(ledPin, LOW);

    Serial.println("Kondisi Aman");
  }

  Serial.println("-------------------------");

  delay(2000);
}
```

Penjelasan Logika

Program bekerja dengan menginisialisasi pin D6 (Relay) dan D1 (LED) sebagai output yang dimatikan saat awal *booting* untuk mencegah korsleting atau aktivasi aktuator yang tidak disengaja. Di dalam fungsi loop(), mikrokontroler melakukan pengambilan data bersamaan dari pin A0 (analogRead) dan pin D7 (readTemperature). Jika data sensor terkonfirmasi valid (bukan *Not-a-Number*), data diumpankan ke *rule engine* yang menggunakan operator logika OR (||). Logika ini memeriksa dua ambang batas: apakah suhu melebihi 34 °C atau nilai ADC LDR kurang dari 300. Jika salah satu atau kedua kondisi tersebut bernilai benar (true), mikrokontroler mengirim sinyal untuk mengaktifkan Relay dan menyalakan LED serta mencetak status peringatan; sebaliknya, jika kedua kondisi bernilai salah (false), aktuator dimatikan dan status aman ditampilkan pada Serial Monitor.

Hasil Pengujian

Saat sistem berada pada kondisi normal dengan suhu 34 °C atau kurang dan nilai LDR 300 atau lebih, Serial Monitor menampilkan parameter lingkungan secara stabil dengan status "Status: Kondisi Aman" sementara Relay dan LED tetap mati, namun ketika diuji pada kondisi panas di atas 34 °C maupun kondisi gelap di bawah 300, operator logika OR mengevaluasi kondisi menjadi *true* sehingga Relay dan LED langsung aktif serta mencetak status "Status: Peringatan: Aktuator Aktif!", dan saat kabel komunikasi sensor DHT dilepas sengaja untuk mensimulasikan kegagalan koneksi, fungsi isnan() secara efektif mencegah pemrosesan data yang rusak dan menampilkan pesan *error* tanpa menghentikan eksekusi program.

